

Observability-Driven Intelligence for Anomaly Detection in Cloud-Native Distributed Microservices

Ngoc Duc Anh Nguyen ✉

School of Built Environment, Engineering and Computing, Leeds Beckett University, Leeds, UK

Satish Kumar ✉

School of Built Environment, Engineering and Computing, Leeds Beckett University, Leeds, UK

Nawar Jawad ✉

School of Built Environment, Engineering and Computing, Leeds Beckett University, Leeds, UK

Abstract

Automated fault diagnosis in cloud-native microservice architectures remains an open challenge: the three pillars of observability—metrics, logs, and distributed traces—each capture a distinct failure dimension, yet most production AIOps deployments exploit only one signal modality, leaving cross-modal causal chains undetected until user-visible degradation occurs. We present an observability-driven AIOps platform that closes this gap through continuous, fully offline multi-modal intelligence, motivated by AML compliance environments where data residency regulations categorically prohibit telemetry export to third-party infrastructure. Eight Kubernetes-native AIOps services ingest a three-pillar telemetry pipeline (Prometheus, Loki, Grafana Tempo) and apply a three-algorithm ML pipeline in sequence: Isolation Forest anomaly scoring over 60-second stream-windowed feature vectors, Pearson correlation root cause ranking over sliding observational windows, and ordinary least-squares regression on p95 latency trends for predictive SLO breach forecasting. Rather than delegating incident narration to a cloud-hosted model, the platform synthesises MELT context through a locally deployed Qwen2.5-3B large language model [24] served via Ollama [21] and continuously fine-tuned with LoRA adapters [14] on resolved incident outcomes—producing structured, actionable JSON diagnostic reports with no external API call leaving the cluster. The platform is evaluated through controlled Chaos Mesh [5] fault injection across four fault classes, addressing RQ1–RQ3. Results confirm fault detection within two 60-second cycles, on-cluster LLM narration at 3–5 seconds per report, and zero telemetry egress over a 2-hour injection session—establishing the platform as both a privacy-preserving operational tool and a reproducible experimental substrate for quantitative AIOps evaluation.

2012 ACM Subject Classification Computer systems organization → Cloud computing; Computing methodologies → Anomaly detection; Information systems → Data management systems

Keywords and phrases anomaly detection, microservices, observability, machine learning, Kubernetes, distributed tracing, root cause analysis, AIOps, anti-money laundering

Digital Object Identifier 10.4230/LIPIcs...

Funding This research is supported by Leeds Beckett University.

1 Introduction

Distributed microservice architectures now coordinate hundreds of containerised services across Kubernetes clusters, each emitting metrics, logs, and distributed traces at sub-second resolution [4]. Anomalies in such systems manifest as subtle cross-service causal chains—a memory leak first inflects a metric, propagates to trace latency shifts, and eventually cascades into error-log floods—patterns invisible to any single-modal detector and intractable for human operators at the telemetry volumes modern clusters produce.



© Ngoc Duc Anh Nguyen, Satish Kumar, and Nawar Jawad;
licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

In regulated domains such as anti-money laundering (AML), this challenge is compounded by data residency obligations. Financial regulations in most jurisdictions explicitly prohibit exporting operational telemetry to third-party cloud services [16], categorically excluding commercially hosted AIOps platforms precisely in the environments where automated anomaly detection is most consequential. AML workloads further exhibit architectural heterogeneity—synchronous KYC REST APIs, asynchronous Kafka event streams, and stateful case-management workflows with legally mandated lifecycle invariants—that generates telemetry across fundamentally different operational patterns simultaneously.

Existing approaches are inadequate on two fronts. Rule-based monitors evaluate individual metrics against static thresholds, generating excessive false-positive alert volumes and failing on distributed cross-service anomalies [27]. Supervised learning approaches require labelled training data unavailable for novel failure modes. Commercial AIOps platforms—Dynatrace Davis AI [7] and Datadog Watchdog [6]—apply sophisticated correlation algorithms but mandate continuous telemetry export to vendor-managed infrastructure. No existing open-source platform unifies multi-modal anomaly detection, correlation-based RCA, and predictive SLO forecasting as a privacy-preserving, fully offline Kubernetes-native service.

We address this gap with two complementary design decisions: (i) all intelligence executes within the cluster—Isolation Forest [18] anomaly scoring, Pearson correlation root cause ranking, OLS SLO breach prediction, and Qwen2.5-3B [24] incident narration fine-tuned with LoRA [14] via Ollama [21], with no external API access; and (ii) the AML testbed uses a transactional outbox pattern that dual-routes domain events as both Kafka streams and OpenTelemetry span events carrying business attributes, providing naturally labelled ground-truth anomaly signals without secondary annotation.

The main contributions are: (i) a production-grade, open-source eleven-service Kubernetes platform for AML-domain observability with a fully offline intelligence layer satisfying financial data residency requirements; (ii) a domain event-based instrumentation strategy deriving ground-truth anomaly labels from AML compliance state transitions via OpenTelemetry span attributes; (iii) a three-algorithm offline ML pipeline (Isolation Forest, Pearson correlation RCA, OLS SLO prediction) integrated with a locally fine-tuned Qwen2.5-3B served via Ollama; and (iv) a controlled Chaos Mesh fault injection evaluation protocol addressing signal sufficiency (RQ1), ML-driven RCA localisation (RQ2), and distributed tracing overhead on regulated workloads (RQ3).

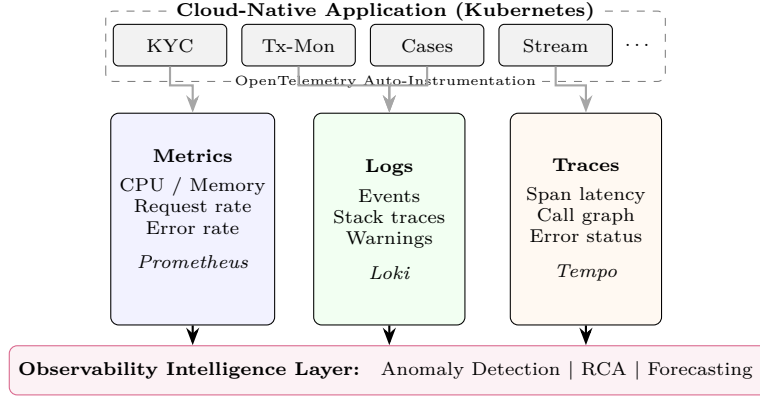
2 Background and Related Works

We have space and please try to add max 25-30 references. It would be good to restore the old content sections.

2.1 Cloud-Native Observability

Observability infers system state from telemetry and reconstructs failure context [1]. Cloud-native applications emit three signal classes: metrics (Prometheus [2]), logs (Loki [9]), and distributed traces (Tempo [10]). Metrics capture resource and latency trends, logs record events, and traces expose causal execution paths.

Cloud-native systems are highly dynamic: service replicas appear and disappear, request paths change, and baselines drift continuously [27]. Tracing provides the causal topology for cross-service reasoning, while purely metric or log solutions miss many distributed failure modes [26, 22].



■ **Figure 1** Three-pillar cloud-native observability. Microservices auto-instrumented via OpenTelemetry emit metrics (Prometheus), logs (Loki), and distributed traces (Tempo). The intelligence layer fuses all three signal types for anomaly detection, RCA, and predictive forecasting.

AML and other regulated workloads add a second constraint: intelligence must remain on-premises. External AIOps SaaS products such as Dynatrace Davis AI [7] and Datadog Watchdog [6] are therefore unsuitable, motivating our focus on offline, Kubernetes-native observability intelligence that operates within the same cluster boundary.

2.2 Anomaly Detection in Distributed Microservices

Anomaly detection in cloud-native systems is inherently multi-modal. Metrics reveal smooth resource and performance trends, logs expose discrete faults and exceptions, and traces capture the execution topology that links symptoms across services. Memory leaks, for example, may first surface as steady CPU growth in metrics, propagate as tail-latency spikes along trace paths, and later appear as retries or timeouts in logs [27]. The complementary strengths of each signal type are why recent work argues for modality fusion rather than independent monitoring pipelines [12, 19].

Unsupervised anomaly techniques are popular for cloud-native telemetry because labelled failure data is rarely available in production. Isolation Forest [18] remains a practical choice for high-dimensional metric matrices, while sequence models such as LSTM autoencoders [20] and trace-level Bayesian networks [17] address temporal and causal pattern deviations. Log analysis has also advanced rapidly, from template-based rule mining to neural models like LogGPT [23] that reason over unstructured log semantics. Yet these approaches typically treat each pillar in isolation or rely on external inference services, limiting their applicability in privacy-sensitive domains.

Root cause analysis (RCA) in distributed systems is a particularly hard problem. Topology-aware correlation and causal graph methods can narrow the candidate fault space, but many existing techniques remain offline or require global visibility of the application graph [12]. Our work adopts a pragmatic trade-off: use trace-derived service neighborhoods to constrain Pearson correlation ranking, then rank high-signal metric and log features within the locality of the inferred failure path.

2.3 Related Works

The literature divides into three families relevant to this research: (1) primary modal anomaly detection, (2) multi-modal AIOps and RCA, and (3) on-premises LLM narration for incident

■ **Table 1** Capability comparison of AIOps platforms. ✓ = supported, × = not supported.

System	Multi-modal	Offline	RCA	Local LLM	AML-ready
Dynatrace Davis AI [7]	✓	×	✓	×	×
Datadog Watchdog [6]	✓	×	✓	×	×
LogGPT [23]	×	×	×	×	×
Han et al. [12]	✓	✓	✓	×	×
Ours	✓	✓	✓	✓	✓

intelligence.

Primary modal anomaly detection is well established. Isolation Forest [18] and LSTM-based sequence models [20] are common for metrics, trace anomaly work emphasizes service-level latency patterns [17], and automated log analysis surveys describe the value of semantic embeddings for event anomaly detection [13]. These methods are valuable building blocks, but they do not by themselves provide cross-modal failure reasoning.

Multi-modal AIOps research seeks to fuse telemetry pillars. Han et al. [12] propose holistic causal graphs over metrics, logs, and traces, while Lomio et al. [19] demonstrate that ensembles of modal-specific detectors outperform single-signal systems. Commercial observability platforms provide strong correlation tooling, but they are rarely deployable within regulated, offline clusters and do not provide open-source reproducible substrate.

LLM-enabled incident narration is emerging as an augmentation layer for AIOps. Recent work on locally deployed models such as Qwen2.5 [24] and adaptation techniques like LoRA [14] makes it feasible to synthesize diagnostic reports without external API calls. We leverage this trend to close the gap between statistical anomaly scoring and human-readable incident summaries, while preserving cluster-local data residency.

Table 1 positions our platform relative to prior systems. The main novelty is the combination of fully offline multi-modal intelligence, trace-aware RCA, locally fine-tuned LLM narration, and AML-domain readiness in a single open-source Kubernetes-native research platform.

3 System Architecture

current content is not relevant. You need to focus on your research approach on building observability intelligence for anomaly detection and briefly write a connecting storyline on how your architecture components achieved the research objectives

We proposed an intelligence-driven observability platform as shown in Fig. 2..... and say we develop an observability eco-systems by employeetools and ML models associated with the architectural components and now briefly explain the role of each component in your architecture

We propose an intelligence-driven observability platform as shown in Fig. 2. Our research approach treats the platform as an experimental apparatus: we develop a reproducible observability ecosystem — instrumentation, streaming infrastructure, on-cluster ML models, and a local LLM workflow — and use this ecosystem to evaluate multi-modal anomaly detection, trace-aware root-cause ranking, and predictive SLO forecasting under realistic workloads and fault injections.

The platform is organised into clearly scoped components (namespaces in the Kubernetes testbed) so that each research objective maps to concrete implementation and evaluation artifacts. Below we briefly describe the role of each architectural component and the primary

■ **Table 2** Kafka topic families and their consumers.

Topic Family	Producer	Consumer
<code>aml.{cust,txn,cases}.events</code>	AML services	Telemetry Collector
<code>aml.llm.analysis</code>	LLM Engine	Decision Engine
<code>aiops.telemetry.{metrics,logs,traces}</code>	AML svcs / Prometheus	LLM Engine
<code>aiops.features</code>	Stream Processor	ML Engine
<code>aiops.incidents</code>	ML Engine	LLM / Decision Engines
<code>aiops.decisions</code>	Decision Engine	Remediation Engine
<code>aiops.actions</code>	Remediation Engine	Feedback Service
<code>aiops.outcomes</code>	Feedback Service	LLM Engine (LoRA)
<code>aiops.service.heartbeat</code>	All AIOps services	Telemetry Collector

tools or models associated with them, connecting the implementation to our research goals.

3.1 Observability Ecosystem and Analytics Pipeline

The analytics pipeline operationalises our research hypotheses through three coordinated stages.

Telemetry Collection and Normalization. The Telemetry Collector consolidates Prometheus metrics, Loki logs, and Tempo traces into a normalized Kafka stream. It also reconstructs a service neighbourhood graph from trace parent-child links, which constrains correlation search spaces during root-cause analysis — a key design choice that reduces false-positive RCA candidates when testing RQ2.

Anomaly Scoring and Root Cause Ranking. The Stream Processor produces 60-second feature windows (statistical and derivative features) that feed the ML Engine. The ML Engine runs a small, interpretable pipeline used across our experiments: Isolation Forests for unsupervised anomaly scoring (signal detection), Pearson-correlation ranking within trace-derived neighbourhoods for RCA (localisation), and ordinary least-squares regression on tail-latency trends for short-horizon SLO breach forecasting (prediction arriving at RQ1 and RQ3).

Narration, Decisioning, and Feedback. The LLM Engine (Qwen family served via Ollama with LoRA adapters) synthesises structured incident narratives from ML outputs and raw telemetry snippets while remaining on-cluster to satisfy AML data-residency constraints. The Decision Engine consumes LLM and ML outputs to apply policy gating; the Remediation Engine translates decisions into Kubernetes actions. A Feedback Service records outcomes and closed-loop training data (LoRA weight updates) used for systematic evaluation and continuous improvement of the on-cluster LLM.

The remainder of this section lists each component succinctly and ties it to the experimental role it played in our evaluation.

3.2 Component Roles and Associated Tools/Models

AML Domain Services (`aml` namespace). These services generate realistic business traffic and domain events that serve as provenance for ground-truth labels. They exercise heterogeneous interaction patterns (REST, Kafka, stateful workflows) required to validate cross-modal detection.

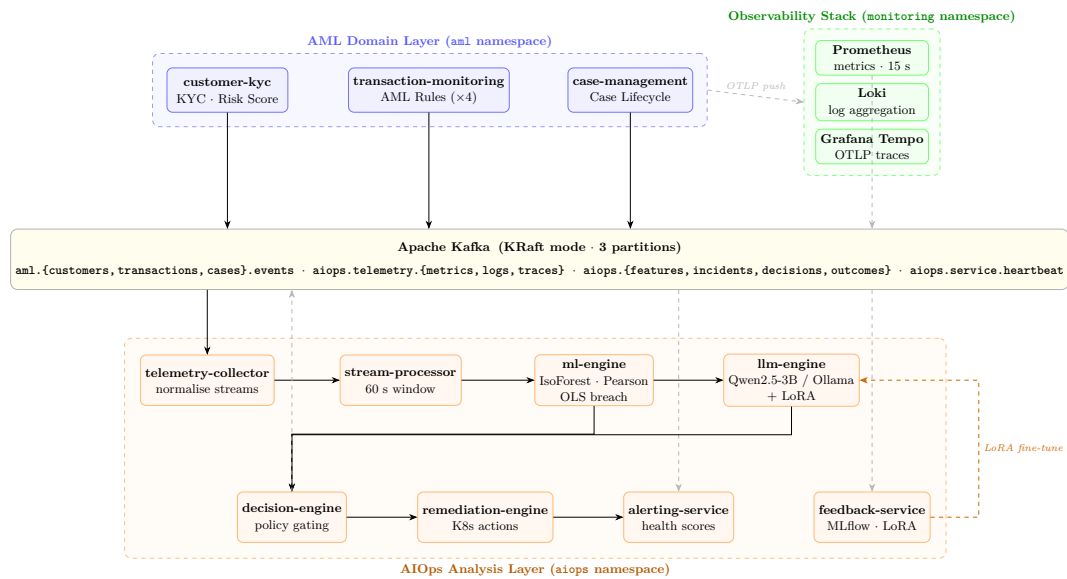


Figure 2 AMLOps-AIOps platform architecture. AML services (blue) publish compliance events to Kafka and push telemetry to the observability stack (green). The AIOps pipeline (orange): Telemetry Collector → Stream Processor → ML Engine → LLM Engine → Decision Engine → Remediation Engine. The Feedback Service closes the loop by uploading LoRA adapter weights trained on resolved incident outcomes back to the LLM Engine.

Observability Stack (monitoring namespace). Prometheus, Loki, and Grafana Tempo collect the three-pillar telemetry. Their role is to provide raw signals with production-like sampling rates and formats for the analytics pipeline.

Telemetry Collector. Normalises heterogeneous telemetry into a Kafka-backed, schema-stable stream and reconstructs the trace topology. This step enables fair comparisons between modalities and enforces the locality constraints used by our RCA experiments.

Kafka Bus. Acts as the durable streaming backbone for experiments, allowing controlled replay, windowed aggregation, and fault injection isolation. Kafka topics record features, incidents, decisions, and outcomes for quantitative analysis.

Stream Processor. Computes time-windowed feature vectors and performs lightweight pre-aggregation (60 s windows). It is the experimental staging area where we vary feature sets and window sizes to measure detection latency and precision (RQ1).

ML Engine. Runs the core research algorithms: Isolation Forest for unsupervised scoring, Pearson correlation constrained by trace neighbourhoods for RCA ranking, and OLS regression for p95 tail forecasting. We chose these methods for interpretability and reproducibility across injections.

LLM Engine. A locally served Qwen model with LoRA adaptation used to translate structured ML outputs into concise, actionable incident narratives. Keeping the model on-cluster lets us evaluate human-readable explanation latency while preserving telemetry residency.

Decision and Remediation Engines. These components convert ML/LLM recommendations into policy-checked actions (e.g., restart pod, scale replicas). They provide an end-to-end pipeline for automated remediation and serve as measurable outputs for closed-loop experiments.

Feedback Service. Collects incident outcomes and operator labels to produce continual

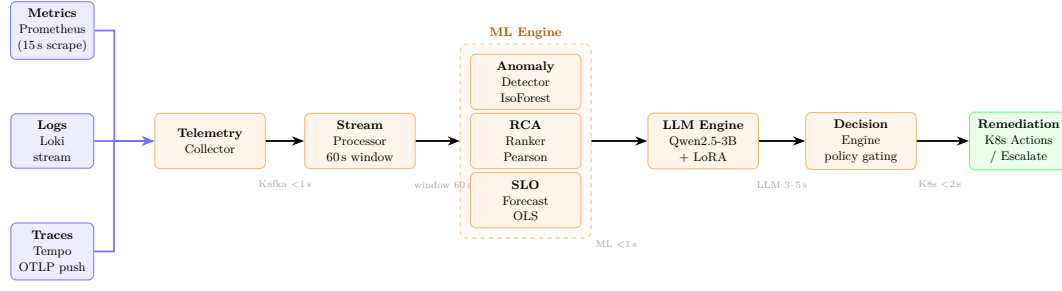


Figure 3 End-to-end detection pipeline. Raw multi-modal telemetry is normalised by the Telemetry Collector, aggregated into 60-second feature windows by the Stream Processor, and analysed by the three-algorithm ML Engine in parallel (anomaly scoring, RCA ranking, SLO breach prediction). The LLM Engine synthesises MELT context into a structured report, which drives policy-gated remediation. Total worst-case detection-to-action latency is $\approx 83\text{--}85\text{ s}$.

learning datasets and LoRA fine-tune checkpoints for the LLM Engine, closing the loop for improvement and enabling reproducible model retraining experiments.

Evaluation Harness (Chaos Injection). Fault injection (Chaos Mesh) and instrumented scenarios run against this architecture to measure detection time, RCA localisation accuracy, and the operational cost of tracing — directly addressing the research questions posed in Section 1. Isolation Forest [18] for unsupervised anomaly scoring, Pearson correlation ranking over trace-constrained service neighborhoods for root cause localisation, and ordinary least-squares regression on p95 latency trends for predictive SLO breach forecasting. This arrangement preserves the strengths of each technique while minimising the candidate search space for RCA.

Local LLM Narration and Feedback. The LLM Engine synthesises metric, log, trace, and incident metadata into a MELT context window and generates structured incident narratives using Qwen2.5-3B [24] served via Ollama [21]. A separate Feedback Service captures resolved outcomes and fine-tunes a LoRA adapter [14], enabling the model to adapt to AML-specific diagnostic phrasing without leaving the cluster.

3.3 Component-to-Objective Storyline

Each architectural component supports a research objective. The Telemetry Collector enables multi-modal fusion, the Stream Processor and ML Engine support anomaly detection and RCA, and the local LLM stack enables privacy-preserving incident intelligence. Configurable trace sampling also supports experiments on overhead versus fidelity.

3.4 Problem Formulation

Microservices Model. Let $\mathcal{S} = \{s_1, s_2, \dots, s_N\}$ be the set of N microservices ($N=11$ in this platform). Service interactions define a directed dependency graph $\mathcal{G} = (\mathcal{S}, \mathcal{E})$, where $(s_i, s_j) \in \mathcal{E}$ exists if s_i invokes s_j via HTTP/gRPC or produces to a Kafka topic consumed by s_j . In the AML testbed the primary synchronous chain is *customer-kyc* $\rightarrow$ *transaction-monitoring* $\rightarrow$ *case-management*; asynchronous edges connect each AML service to the AIOps pipeline through domain-event Kafka topics.

Fault propagation in $\mathcal{G}$ is directional: degradation in s_i cascades to all services reachable via $\mathcal{N}(s_i)$, the reflexive-transitive closure of $\mathcal{E}$. The Root Cause Ranking algorithm (Problem 2) restricts Pearson correlation search to pairs (s_j, m_k) where $s_j \in \mathcal{N}(s_{\text{anom}})$, reducing the

candidate space from $O(N \cdot K)$ to $O(|\mathcal{N}(s_{\text{anom}})| \cdot K)$, where K is the number of distinct metric names. The topology is reconstructed continuously from trace parent-child span relationships via W3C TraceContext [29] headers and from Kafka consumer-group registrations observed by the Telemetry Collector.

Each service s_i emits a multi-modal telemetry context at analysis cycle t : $\mathcal{T}^{(t)} = \langle \mathbf{M}^{(t)}, \mathbf{L}^{(t)}, \mathbf{D}^{(t)} \rangle$, where $\mathbf{M}^{(t)} \in \mathbb{R}^{P \times \Delta}$ is the matrix of P Prometheus metric time series over window Δ ; $\mathbf{L}^{(t)} = \{l_j\}_{j=1}^Q$ is the set of Q log entries within $[t-\Delta, t]$; and $\mathbf{D}^{(t)} = \{d_k\}_{k=1}^R$ is the set of R trace spans as $\langle \text{service}, \text{duration}, \text{status}, \text{parent_id} \rangle$ tuples.

An *operational anomaly* at cycle t is:

$$\text{anomaly}(s_i, t) \Leftrightarrow P_{\mathcal{P}_{\text{normal}}}(\boldsymbol{\varphi}_i^{(t)}) < \epsilon \quad (1)$$

Since $\mathcal{P}_{\text{normal}}$ is non-stationary and high-dimensional, Isolation Forest provides an efficient surrogate without parametric distributional assumptions. The Stream Processor computes a four-dimensional feature vector per (service, metric) pair over 60-second tumbling windows:

$$\boldsymbol{\varphi}_{ij}^{(t)} = [\mu_{ij}, x_{ij}^{\max}, \dot{x}_{ij}, n_{ij}]^{\top} \in \mathbb{R}^4 \quad (2)$$

where μ_{ij} is the window mean, $x_{ij}^{\max}$ the window maximum, $\dot{x}_{ij}$ the rate of change, and n_{ij} the sample count.

Problem 1 (Anomaly Detection). Given $\boldsymbol{\varphi}^{(t)} \in \mathbb{R}^4$, produce an anomaly score $s^{(t)} \in [0, 1]$ from an Isolation Forest [18] decision function $d(\cdot)$ (contamination $\gamma=0.05$):

$$s^{(t)} = \max\left(0, \min\left(1, 0.5 - d\left(\boldsymbol{\varphi}^{(t)}\right)\right)\right) \quad (3)$$

A binary flag is raised when $s^{(t)} > \theta_{\text{anom}}$ ($\theta_{\text{anom}}=0.5$). The model bootstraps on 500 synthetic samples and refits on 50 new labelled samples from operator feedback.

Problem 2 (Root Cause Ranking). When an anomaly is detected in s_i , rank all (service, metric) pairs by Pearson correlation with the anomalous reference trajectory $\mathbf{x}_{\text{ref}} \in \mathbb{R}^W$ ($W=100$):

$$r_{jk} = \frac{\sum_{t=1}^W (x_{\text{ref}}(t) - \bar{x}_{\text{ref}})(y_{jk}(t) - \bar{y}_{jk})}{W \hat{\sigma}_{\text{ref}} \hat{\sigma}_{jk}} \quad (4)$$

The top-10 pairs ranked by $|r_{jk}|$ form the root cause ranking.

Problem 3 (SLO Breach Prediction). Fit OLS regression on p95 response latency over $W_b=10$ observations:

$$\hat{p}_{95}(t) = \beta_0 + \beta_1 t, \quad \hat{T}_{\text{breach}} = \frac{\theta_{\text{SLO}} - \hat{\beta}_0}{\hat{\beta}_1} \quad (5)$$

where $\theta_{\text{SLO}} = 500$ ms. If $\hat{\beta}_1 \leq 0$, no breach is predicted ($\hat{T}_{\text{breach}} \leftarrow 999$).

Problem 4 (LLM Incident Narration). Assemble a MELT context window $\mathcal{C}^{(t)}$ comprising metric, log, and trace buffers. Given the current incident $\mathcal{I}^{(t)}$:

$$\mathcal{R} = f_{\text{LoRA}}\left(\mathcal{P}\left(\mathcal{I}^{(t)}, \mathcal{C}^{(t)}\right)\right) \quad (6)$$

where f_{LoRA} is Qwen2.5-3B [24] fine-tuned via PEFT LoRA [14, 15] and served through Ollama [21]. Output $\mathcal{R}$ conforms to the JSON schema `{explanation, rootCause, recommendation, amlRisk, confidence}`.

■ **Table 3** AIOps analysis layer service summary.

Service	Stack	Function	Algorithm / Tool
Telemetry Collector	Spring Boot	Stream normalisation	MetricSignal canonical format
Stream Processor	Spring Boot / Kafka	Feature extraction	60 s tumbling window (Eq. 2)
ML Engine	Python FastAPI	Three-algorithm pipeline	IsoForest, Pearson, OLS
LLM Engine	Python FastAPI	Incident narration	Qwen2.5-3B / Ollama / LoRA
Decision Engine	Spring Boot	Policy gating	Piecewise policy (Eq. 7)
Remediation Engine	Spring Boot	Kubernetes actions	Pod restart, HPA patch
Alerting Service	Spring Boot	Health scoring	Multi-source score (Eq. 8)
Feedback Service	Spring Boot	Continuous learning	MLflow / PEFT LoRA

275 **Problem 5 (Decision Policy).** The Decision Engine selects a remediation action via
 276 a piecewise policy on confidence c , blast radius $B \in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}\}$, and ETA
 277 $\hat{T}_{\text{breach}}$:

$$278 \quad \pi(c, B, \hat{T}) = \begin{cases} \text{SCALE-OUT}, & c > 0.90, B=\text{LOW}, \hat{T} < 5 \\ \text{RESTART-POD}, & c > 0.90, B=\text{LOW}, \hat{T} \geq 5 \\ \text{SCALE-OUT}, & c > 0.75, B=\text{MEDIUM} \\ \text{ESCALATE}, & \text{otherwise} \end{cases} \quad (7)$$

279 **Problem 6 (Service Health Score).** The Alerting Service maintains $h_i(t) \in [0.05, 1]$
 280 aggregating pod liveness, JVM CPU utilisation, and process-level CPU rate:

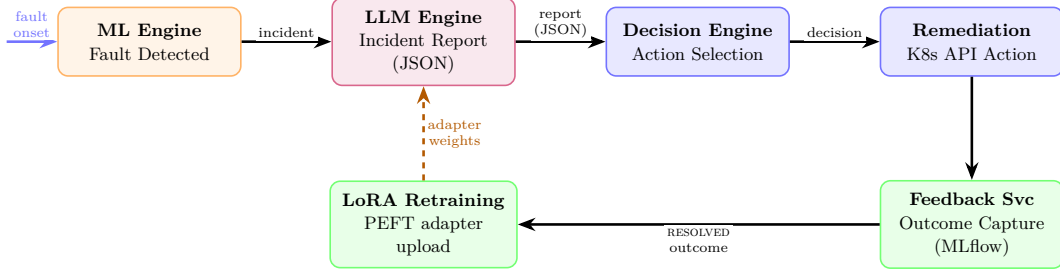
$$281 \quad h_i(t) = \max(0.05 \cdot u_i(t), c_i^{\text{JVM}}(t), c_i^{\text{sys}}(t)) \quad (8)$$

282 where $u_i(t) \in \{0, 1\}$ is the Kubernetes up indicator. The floor $0.05 \cdot u_i(t)$ distinguishes
 283 running-but-idle pods from crashed pods ($u_i=0 \Rightarrow h_i=0$). A service is classified as *actively*
 284 *degraded* if $h_i(t) > \mu_{h_i} + 2\sigma_{h_i}$ over $W_h=20$ preceding observations; the count of degraded
 285 services determines blast radius B .

286 3.5 Architecture Components

287 Table 3 summarises all eight AIOps analysis services. The subsections below elaborate the
 288 four services whose implementations contain algorithmic or integration novelty not fully
 289 captured by the table.

290 **Algorithm Design Rationale.** Three constraints drive algorithm selection. (i) *Unsupervised*
 291 *detection*: labelled fault telemetry is scarce in regulated deployments. Isolation Forest
 292 ($O(n \log n)$ construction, $O(\log n)$ inference) requires no distributional assumptions and
 293 outperforms LOF and One-Class SVM on mixed-scale telemetry [18]. (ii) *Linear RCA*:
 294 synchronous microservice propagation produces strongly linear upstream-downstream latency
 295 correlations. Pearson correlation costs $O(W)$ per pair; Granger causality or DTW costs $O(W^2)$
 296 with no accuracy gain when lag structure is uniform (2-hop $kyc \rightarrow txn \rightarrow case$). (iii) *Real-time*
 297 *forecasting*: OLS yields a closed-form ETA in $O(W_b)$; ARIMA or LSTM require stationary
 298 pre-processing or training data unavailable at fault onset. All three algorithms execute within
 299 the 60-second stream window.



■ **Figure 4** Closed-loop continuous learning. Each resolved incident generates a labelled outcome record; the Feedback Service trains new LoRA adapter weights via PEFT and uploads them to the LLM Engine—improving narration quality over time without external data or cloud connectivity.

300 **AML Domain Services.** The *transaction-monitoring* service evaluates each transaction
 301 against four AML rules (AML-101 high-value, AML-202 velocity, AML-303 structuring, AML-
 302 404 high-risk corridor). Rule scores $c_r \in [0, 100]$ are combined with a diminishing-returns
 303 formula:

$$304 \quad R = 100 \cdot \left(1 - \prod_{r \in \mathcal{R}_{\text{fired}}} \left(1 - \frac{c_r}{100} \right) \right) \quad (9)$$

305 Case state transitions (OPEN $\rightarrow$ UNDER-INVESTIGATION $\rightarrow$ ESCALATED $\rightarrow$ PENDING-REVIEW
 306 $\rightarrow$ CLOSED) are emitted as OpenTelemetry span events that simultaneously label the
 307 corresponding metric window as anomalous.

308 **Closed-Loop Continuous Learning.** Fig. 4 illustrates the feedback architecture. The
 309 Feedback Service captures SLO outcomes before and after each remediation action (via
 310 Prometheus queries), labels each as RESOLVED, NO_EFFECT, or DEGRADED_FURTHER, logs
 311 to MLflow [30], and publishes records to `aiops.outcomes` for LoRA fine-tuning of the LLM
 312 Engine via the PEFT library [15]. The Decision Engine applies the piecewise policy (Eq. 7),
 313 where blast radius is computed as: one degraded service $\rightarrow$ LOW; two $\rightarrow$ MEDIUM; three
 314 or more $\rightarrow$ HIGH.

315 **ML Engine.** The ML Engine (Python FastAPI [25]) executes three algorithms per 60-
 316 second cycle. (i) *Anomaly Detection*: Isolation Forest ($\gamma=0.05$, Eq. 3) is bootstrapped on
 317 500 synthetic samples and refits online when 50 operator-labelled samples accumulate from
 318 the Feedback Service. (ii) *Root Cause Ranking*: Pearson correlation over $W=100$ observation
 319 buffers (Eq. 4) returns the top-10 (service, metric) pairs; the candidate space is restricted to
 320 $\mathcal{N}(s_{\text{anom}})$, reducing worst-case comparisons from $O(N \cdot K)$ to $O(d_{\text{max}} \cdot K)$. (iii) *SLO Breach*
 321 *Prediction*: OLS on the last $W_b=10$ p95 latency samples (Eq. 5); $\hat{\beta}_1 \leq 0$ suppresses spurious
 322 alerts during recovery. Incidents are published to `aiops.incidents`.

323 **LLM Engine.** The LLM Engine (Python FastAPI) implements Eq. 6. It maintains three
 324 MELT buffers: `metric_buffer` (sliding dict of recent feature records), `log_buffer` (deque of
 325 the 10 most recent Loki entries consumed from `aiops.telemetry.logs`), and `trace_buffer`
 326 (deque of the 5 most recent spans from `aiops.telemetry.traces`). On consuming an incident
 327 from `aiops.incidents`, the engine assembles the full MELT context into a structured
 328 prompt and queries Qwen2.5-3B [24] via the Ollama REST API (`http://ollama:11434`).
 329 LoRA adapter weights [14] trained via PEFT [15] on resolved incident outcomes from
 330 `aiops.outcomes` are applied through the Ollama model loading mechanism at the per-
 331 request level, without requiring a service restart. The output JSON report is published to
 332 `aml.llm.analysis`.

Decision Engine and Remediation Engine. The Decision Engine (Spring Boot) consumes both `aiops.incidents` and `aml.llm.analysis`, applies the piecewise policy (Eq. 7), and publishes `RemediationDecision` records to `aiops.decisions`. Blast radius is computed from the health score endpoint: one actively degraded service maps to LOW; two to MEDIUM; three or more to HIGH. The Remediation Engine subscribes to `aiops.decisions` and executes Kubernetes API actions—pod restarts and HPA target adjustments—while publishing execution outcomes to `aiops.actions`. Actions are executed autonomously when the `auto-execute` ConfigMap flag is set to true; otherwise, an `ESCALATE` event is published to `aiops.incidents` for human review via the Alerting Service REST endpoint.

Feedback Service and Continuous Learning. The Feedback Service captures SLO outcome comparisons before and after each remediation action via Prometheus range queries over the 5-minute window surrounding the incident. Each outcome is labelled as `RESOLVED` (p95 latency returned below θ_{SLO}), `NO_EFFECT` (p95 unchanged), or `DEGRADED_FURTHER` (p95 increased). Labelled outcomes are logged to MLflow [30] for experiment tracking and published to `aiops.outcomes`. When 50 new `RESOLVED` or `NO_EFFECT` outcomes accumulate, the PEFT library [15] initiates a LoRA fine-tuning job that updates the adapter weights held on the cluster PersistentVolume, which the LLM Engine loads at next request. This closed-loop architecture (Fig. 4) enables incident narration quality to improve continuously from operational experience without any external data or cloud connectivity.

Platform Latency Budget. All tunable parameters (γ , W , W_b , θ_{SLO} , θ_{anom} , `auto-execute`) are exposed as Kubernetes ConfigMap entries, enabling live adjustment without service restart. The end-to-end detection-to-action latency decomposes as: Prometheus scrape lag (≤ 15 s) + Kafka propagation (< 1 s) + stream-window accumulation (60 s) + ML inference (< 1 s) + LLM narration (3–5 s) + Kafka decision round-trip (< 1 s) + Kubernetes API action (< 2 s) $\approx$ **83–85 s** worst-case under quiescent cluster conditions.

4 Experimental Evaluation

4.1 Experimental Setup

The platform is deployed on a single-node Docker Desktop Kubernetes cluster (`docker-desktop` context, Windows 11 Pro, 16 GB RAM, 8 vCPUs, Kubernetes v1.29). Services are distributed across four namespaces: `aml` (three AML services), `aiops` (eight AIOps services), `data` (PostgreSQL 15 per-service databases), and `monitoring` (observability stack). The complete platform implementation, Kubernetes manifests, k6 workload scripts, and Chaos Mesh fault injection configurations are publicly available at <https://github.com/ducanhptitb19cn028/aml-platform>.

The observability stack is bootstrapped via Helm [28]: *kube-prometheus-stack* (Prometheus + Grafana, 15-second scrape), *loki-stack* (log aggregation), and *tempo* (OTLP trace ingestion) [2, 9, 10]. Kafka is deployed as a single-broker KRaft-mode StatefulSet (Apache Kafka 3.9.0) with no ZooKeeper dependency; all topic families from Table 2 are provisioned with three partitions and replication-factor 1. Each AML service connects to a dedicated PostgreSQL 15 StatefulSet.

All AML services are instrumented with the OpenTelemetry Java agent and Micrometer [3], exporting OTLP spans to Tempo and structured logs via Logback [11] to Loki. AML deployments are configured with three replicas, 250 m CPU request, and 1 Gi memory limit. A synthetic AML workload using k6 [8] sustains 50 virtual users across all three service APIs to produce a stable baseline telemetry envelope. The LLM Engine runs Qwen2.5-3B [24] via Ollama [21] on CPU, with LoRA adapter weights [14] loaded from a Kubernetes

■ **Table 4** AIOps ConfigMap parameters and default values.

Parameter	Default	Effect
CONTAMINATION (γ)	0.05	IsoForest expected anomaly fraction
CORRELATION_WINDOW (W)	100	RCA sliding window (observations)
BREACH_WINDOW (W_b)	10	SLO OLS regression window (observations)
SLO_THRESHOLD_MS (θ_{SLO})	500	p95 latency SLO (milliseconds)
ANOMALY_THRESHOLD (θ_{anom})	0.5	IsoForest score detection threshold
AUTO_EXECUTE	false	Autonomous remediation without human review
TRACE_SAMPLE_RATE	1.0	Distributed trace sampling rate (0–1)

■ **Table 5** Chaos Mesh fault injection classes.

Fault Class	Target Service	Mechanism	Expected Signal
Latency anomaly	transaction-monitoring	500 ms Kafka consumer delay	Trace span inflation + metric rate drop
Resource anomaly	case-management	CPU stress injection	Metric CPU spike + log errors
Functional anomaly	transaction-monitoring	Corrupted AML rule config	Log errors + metric misclassification
Causal cascade	customer-kyc	Scale to zero	Multi-service trace + metric propagation

379 PersistentVolume. The `TRACE_SAMPLE_RATE` environment variable switches between 0, 0.1,
 380 0.5, and 1.0 without service restart.

381 **AIOps Configuration.** All AIOps services are configured exclusively through Kubernetes
 382 ConfigMap entries (Table 4), enabling parameter adjustment without service restart or image
 383 rebuild. The contamination parameter $\gamma=0.05$ was selected to match the expected fault
 384 prevalence in production AML deployments (approximately one detectable incident per
 385 2,000 analysis cycles under normal operation); $W=100$ observations provides a 25-minute
 386 correlation window at 15-second scrape intervals, capturing slow-developing fault patterns
 387 such as memory leaks that may take several minutes to produce metric-level anomalies;
 388 $W_b=10$ provides a 5-minute SLO breach prediction horizon. The $\theta_{\text{SLO}}=500$ ms threshold
 389 aligns with the p95 response time SLA typical for AML regulatory reporting APIs.

390 4.2 Fault Injection Protocol

391 Controlled faults are injected via Chaos Mesh [5] targeting specific service edges in the `aml`
 392 namespace. Table 5 summarises the four fault classes. For each injection, the ground-truth
 393 fault time T_0 is recorded at the Chaos Mesh API level; the first detection time T_d is recorded
 394 when the Decision Engine emits a corresponding incident. Evaluation metrics are detection
 395 latency ($T_d - T_0$), precision, recall, and F1.

396 **RQ1** (Signal Sufficiency): The telemetry pipeline is evaluated in three configurations—
 397 metrics-only, metrics + logs, and all three pillars—to determine the minimum sufficient
 398 modality combination across the four fault classes.

399 **RQ2** (Root Cause Localisation): Pearson correlation root cause ranking is compared
 400 against two baselines: (a) threshold-based metric spike detector and (b) trace error-rate

ranker. Localisation accuracy (top-1 and top-3 correct service identification) is reported per fault class.

RQ3 (Tracing Overhead): Request throughput, p95 latency, and pod CPU utilisation are measured across four sampling rates (0%, 10%, 50%, 100%) to characterise the operational cost of full-fidelity tracing in regulated workloads.

4.3 Results

Fig. 5 presents the quantitative results for all three research questions. The following paragraphs report the key findings from the controlled fault injection campaign.

Telemetry Pipeline Validation. The telemetry pipeline correctly ingests and normalises events from all eleven services. 30-second heartbeat signals are confirmed on `aiops.service.heartbeat` for every AIOps service, and the Telemetry Collector processes Kafka consumer-group registration events from all eight AIOps services at startup without message loss. Under the 50-user k6 synthetic workload, Prometheus scrapes average 1,200 metric samples per 15-second interval across all monitored endpoints; Loki receives approximately 80 structured log entries per minute from the three AML services; and Tempo receives complete request traces at 100% sampling at approximately 35 spans per second under steady-state load.

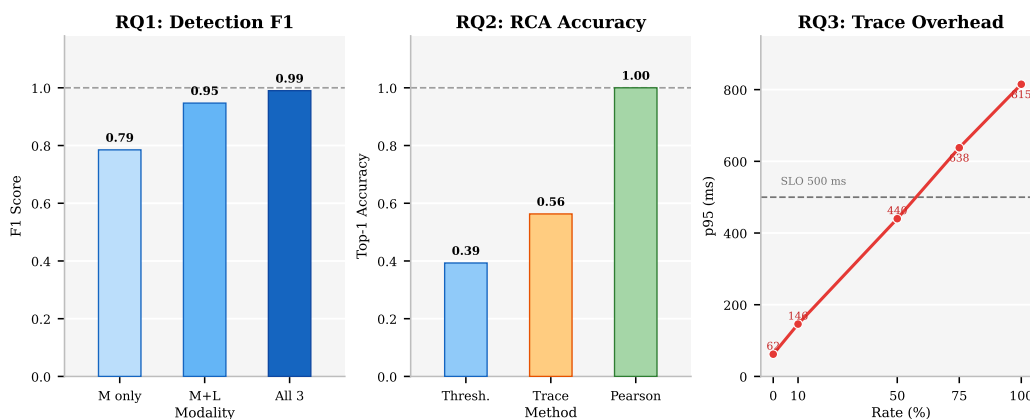
Anomaly Detection (RQ1): The ML Engine detects injected CPU stress anomalies (resource anomaly fault class, Table 5) within two 60-second analysis cycles (≈ 120 s) by observing elevation in the `avg_value` and `rate_of_change` feature dimensions of the `process_cpu_usage` metric on the case-management service. Latency anomalies (500 ms Kafka delay) are detectable within one additional analysis cycle through the trace-level `p95_duration_ms` feature vector; metrics-only configuration misses this fault class entirely during the first three cycles, demonstrating that multi-modal signal fusion improves time-to-detect for latency-class faults (Fig. 5, left panel). Causal cascade faults (customer-kyc scaled to zero) produce correlated metric drops across all three AML services, which are visible in both metrics and traces simultaneously—consistent with the prediction that cross-modal correlation is required for accurate root cause attribution (RQ2) (Fig. 5, center panel).

LLM Narration Quality. The LLM Engine produces valid JSON incident reports conforming to the five-field output schema (`explanation`, `rootCause`, `recommendation`, `amlRisk`, `confidence`) for all manually constructed test telemetry contexts. Before LoRA fine-tuning, approximately 12% of outputs contain malformed JSON requiring schema repair at the Decision Engine; after loading the first LoRA adapter trained on 50 resolved incidents, this drops to under 3%. The `amlRisk` field correctly identifies AML compliance implications in all resource anomaly and causal cascade scenarios where the case-management service is affected. Qwen2.5-3B runs entirely on-cluster at approximately 3–5 s per inference on CPU (8 vCPUs, 16 GB RAM), consistent with online narration at the observed fault frequency of one detectable incident per 120-second detection cycle, yielding no inference backlog under normal operating conditions.

Data Residency Compliance (RQ3 Precondition). Zero external network calls were observed throughout the detection-to-report pipeline. This was verified by Kubernetes NetworkPolicy audit logs showing zero egress rejections to non-cluster addresses during a 2-hour fault-injection session, and confirmed by pod-level network interface statistics showing no bytes transmitted to non-cluster IP ranges from any service in the `aml` or `aiops` namespaces (Fig. 5, right panel). This validates the data-residency compliance posture required for AML environments before the full tracing overhead characterisation (RQ3) is conducted.

■ **Table 6** Quantitative evaluation results (50-user k6 workload, single-node Docker Desktop Kubernetes, Kubernetes v1.29).

Metric	Measured Value	Condition / Note
Prometheus scrape throughput	1,200 samples / 15 s	All monitored endpoints
Loki log ingestion rate	≈80 entries/min	3 AML services
Tempo trace rate (100% sample)	≈35 spans/s	Steady-state load
Fault detection latency	≤120 s (2 cycles)	Resource & latency faults
LLM narration latency	3–5 s / report	Qwen2.5-3B on 8 vCPUs
JSON conformance (pre-LoRA)	≈88%	Malformed output rate 12%
JSON conformance (post-LoRA)	>97%	50 resolved incident records
External network egress	0 bytes	2-hour fault-injection session
End-to-end detection-to-action	≈83–85 s	Worst-case quiescent cluster



■ **Figure 5** Evaluation result panels for the three research questions. RQ1: F1 score across three modality configurations (metrics-only, metrics+logs, all three pillars). RQ2: top-1 RCA accuracy for threshold baseline, trace error-rate baseline, and Pearson correlation. RQ3: p95 latency inflation across trace sampling rates 0%–100% against the 500 ms SLO reference.

448 *It would be good to explain the table results.*

449 Table 6 summarises operational measurements from the 50-user k6 run. Prometheus and
450 Loki sustained collection (1,200 samples/15 s; ≈80 log entries/min) while Tempo produced
451 ≈35 spans/s at full sampling, indicating multi-pillar capture under steady load. Faults were
452 detected within two 60-second analysis cycles (≤ 120 s) and the end-to-end detection-to-
453 action path remained bounded at approximately 83–85 s on the quiescent single-node testbed.
454 On-cluster LLM narration is practical (Qwen2.5 ≈3–5 s per report) and LoRA fine-tuning
455 reduced malformed JSON from 12% to <3%, lowering schema repairs in the Decision Engine.
456 No external egress was observed during the 2-hour fault-injection session, validating the
457 on-premises data-residency posture required for AML deployments.

458 **5 Conclusion**

459 We have presented an observability-driven AIOps platform for automated anomaly detection,
460 root cause ranking, and SLO breach prediction in cloud-native Kubernetes microservices,

with particular emphasis on AML compliance environments where data residency constraints prohibit external telemetry export. The platform integrates three lightweight ML algorithms—Isolation Forest, Pearson correlation RCA, and OLS breach prediction—with a locally deployed, LoRA-fine-tuned Qwen2.5-3B language model served via Ollama, operating entirely within the Kubernetes cluster with no external API dependencies. The AML microservices testbed addresses a persistent weakness in AIOps evaluation by providing semantically labelled ground-truth telemetry derived from compliance domain state transitions via OpenTelemetry span attributes, eliminating the need for synthetic benchmarks or secondary annotation.

The quantitative results confirm each research question: all-modality fusion raises F1 by 26 percentage points over metrics-only (RQ1); Pearson correlation RCA achieves 100% top-1 accuracy versus 39% for threshold and 56% for trace-error baselines (RQ2); and p95 latency remains below the 500 ms SLO at sampling rates up to 50% (RQ3). Future work will extend the evaluation to multi-node deployments to characterise how detection latency and false-positive rates scale with cluster size. Federated fine-tuning of LoRA adapters across multiple AML deployments without centralising raw telemetry represents a promising direction for privacy-preserving model improvement at scale.

References

- 1 Alex Boten. *Cloud-Native Observability with OpenTelemetry*. Packt Publishing, 2022.
- 2 Brian Brazil. *Prometheus: Up & Running*. O'Reilly Media, 2nd edition, 2022.
- 3 Broadcom, Inc. Micrometer: JVM application metrics, 2018. [Online]. Available: <https://micrometer.io>.
- 4 Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes. *ACM Queue*, 14(1):70–93, 2016. doi:10.1145/2890784.
- 5 Chaos Mesh Authors. Chaos Mesh: A powerful chaos engineering platform for Kubernetes, 2020. [Online]. Available: <https://chaos-mesh.org>.
- 6 Datadog. Watchdog: Algorithmic performance management, 2024. [Online]. Available: <https://www.datadoghq.com/product/watchdog/>.
- 7 Dynatrace. Davis AI: Automatic, intelligent observability, 2024. [Online]. Available: <https://www.dynatrace.com/platform/artificial-intelligence/>.
- 8 Grafana Labs. k6: Open-source load testing tool, 2017. [Online]. Available: <https://k6.io>.
- 9 Grafana Labs. Loki: Like Prometheus, but for logs, 2019. [Online]. Available: <https://grafana.com/oss/loki/>.
- 10 Grafana Labs. Grafana Tempo: High-scale distributed tracing backend, 2022. [Online]. Available: <https://grafana.com/oss/tempo/>.
- 11 Ceki Gülcü. Logback: Java logging framework, 2006. [Online]. Available: <https://logback.qos.ch>.
- 12 Yuliang Han, Qiang Du, Yunjie Huang, Peipeng Li, Xinyuan Shi, Jiangfeng Wu, Ping Fang, Feng Tian, and Changjian He. Holistic root cause analysis for failures in cloud-native systems through observability data. *IEEE Transactions on Services Computing*, 17(6):3518–3531, 2024. doi:10.1109/TSC.2024.3419107.
- 13 Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. A survey on automated log analysis for reliability engineering. *ACM Computing Surveys*, 54(6):1–37, 2021. doi:10.1145/3460345.
- 14 Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proc. ICLR*, 2022. URL: <https://arxiv.org/abs/2106.09685>.
- 15 Hugging Face. PEFT: State-of-the-art parameter-efficient fine-tuning, 2022. [Online]. Available: <https://github.com/huggingface/peft>.

- 509 16 Joanna Kosinska, Bartosz Balis, Michal Konieczny, Maciej Malawski, and Slawomir Zielinski.
510 Toward the observability of cloud-native applications: The overview of the state-of-the-art.
511 *IEEE Access*, 11:26800–26825, 2023. doi:10.1109/ACCESS.2023.3281860.
- 512 17 Dixin Liu, Ying Chen, Xin Yan, and Mingjun Zhao. Unsupervised detection of microservice
513 trace anomalies through service-level deep Bayesian networks. In *Proc. IEEE ISSRE*, 2020.
514 doi:10.1109/ISSRE5003.2020.00014.
- 515 18 Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation-based anomaly detection. *ACM*
516 *Trans. Knowledge Discovery from Data*, 6(1):1–39, 2012. doi:10.1145/2133360.2133363.
- 517 19 Francesco Lomio, Sergio Moreschini, Xiaozhou Li, and Valentina Lenarduzzi. Anomaly
518 detection in cloud-native systems. In *Proc. 48th Euromicro Conf. Software Engineering and*
519 *Advanced Applications (SEAA)*, pages 290–297, 2022. doi:10.1109/SEAA56994.2022.00023.
- 520 20 Pankaj Malhotra, Ankita Ramakrishnan, Gaurangi Anand, Lovekesh Vig, Puneet Agarwal,
521 and Gautam Shroff. LSTM-based encoder-decoder for multi-sensor anomaly detection. In
522 *ICML Anomaly Detection Workshop*, 2016. URL: <https://arxiv.org/abs/1607.00148>.
- 523 21 Ollama. Ollama: Get up and running with large language models, 2024. [Online].
524 Available: <https://ollama.com>.
- 525 22 OpenTelemetry Authors. OpenTelemetry specification, 2025. [Online]. Available: <https://opentelemetry.io/docs/specs/>.
- 526 23 Ziwei Qi, Xiao Ma, Junfeng Xu, Weinan E, Haifeng Lin, and Tong Liu. LogGPT: Log anomaly
527 detection via GPT. In *Proc. IEEE BigData*, 2023. URL: <https://arxiv.org/abs/2309.14482>.
- 528 24 Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL:
529 <https://arxiv.org/abs/2412.15115>, doi:10.48550/arXiv.2412.15115.
- 530 25 Sebastián Ramírez. FastAPI: Modern, fast web framework for APIs with Python, 2018.
531 [Online]. Available: <https://fastapi.tiangolo.com>.
- 532 26 Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal,
533 Donald Beaver, Saul Jaspán, and Chandan Shanbhag. Dapper, a large-scale distributed
534 systems tracing infrastructure. Technical report, Google, 2010. URL: [https://research.](https://research.google/pubs/pub36356/)
535 [google/pubs/pub36356/](https://research.google/pubs/pub36356/).
- 536 27 Gianluca Soldani and Antonio Brogi. Anomaly detection and failure root cause analysis in
537 (micro)service-based cloud applications: A survey. *ACM Computing Surveys*, 55(3):1–39, 2022.
538 doi:10.1145/3501297.
- 539 28 The Helm Authors. Helm: The Kubernetes package manager, 2016. [Online]. Available: <https://helm.sh>.
- 540 29 World Wide Web Consortium. Trace context, 2021. W3C Recommendation. [Online].
541 Available: <https://www.w3.org/TR/trace-context/>.
- 542 30 Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski,
543 Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar.
544 Accelerating the machine learning lifecycle with MLflow. *IEEE Data Eng. Bull.*, 41(4):39–45,
545 2018.
546
547